# Social Networks as Directed Container Comonads: A Categorical Framework with Controlled Sheaf Failure

Robin Langer Neil Ghani

July 2026

**Abstract**

We present a categorical framework for social networks built on three interlocking structures: (1) *directed containers* (Ahman–Chapman–Uustalu) model the social graph, where the comonad's `extract` produces a user's feed and `extend` applies feed algorithms across all perspectives; (2) *presheaves* on a category of viewing contexts model permissions, with contravariance capturing the principle that less privilege sees less content; (3) the *sheaf condition* classifies content by consistency guarantee—public posts form a sheaf (all viewers agree), while blocked content and personalised feeds deliberately fail the gluing axiom. The framework is implemented in Haskell (Servant, type-safe API) with invariants verified in sorry-free Lean 4, reusing the container and comonad library from the *trust-boundaries* project. The Haskell types serve double duty: they are the runtime data model *and* the template for Lean theorem statements, with the Curry–Howard correspondence making the translation nearly mechanical.

## Contents

**1 Introduction** **2**

**2 Background: containers and directed containers** **2**

**3 The social graph as a directed container** **3**
3.1 Haskell encoding . . . . . 3
3.2 Lean proof obligations . . . . . 4

**4 Presheaves: the permission layer** **5**
4.1 Haskell encoding . . . . . 5
4.2 Lean formalisation . . . . . 6

**5 The sheaf condition and controlled failure** **6**
5.1 Haskell encoding . . . . . 7

**6 Interaction: comonad meets presheaf** **7**

**7 The minimal social network** **7**

**8 Implementation architecture** **8**

**9 Future directions** **8**

# 1 Introduction

Social networks are widely acknowledged to be hard to build correctly. The difficulty is not any single problem but the *product* of several interacting concerns: graph traversal, access control, consistency, and ranking. We argue that these concerns decompose cleanly along categorical lines:

1. The **feed problem**—computing a personalised view of the graph—is an instance of the comonadic `extend` operation on a directed container.
2. The **permission problem**—who sees what—is the functoriality condition of a presheaf on viewing contexts.
3. The **consistency problem**—where do different viewers agree?—is the sheaf condition, and a well-designed social network controls *exactly where* this condition holds and where it fails.

This paper develops the mathematics, provides a Haskell implementation that mirrors the categorical structure, and identifies the proof obligations that we discharge in Lean 4.

**Relation to prior work.** The directed container library (containers, directed containers, comonad laws, coKleisli composition, and strength) was developed and formalised in the *trust-boundaries* project [3], where proof registries for epistemically honest AI agents were modelled as directed container comonads. We reuse that library wholesale and provide a new instantiation for social graphs.

# 2 Background: containers and directed containers

We recall the definitions from Abbott–Altenkirch–Ghani [1] and Ahman–Chapman–Uustalu [2], following the Lean formalisation in the trust-boundaries library.

**Definition 2.1** (Container)**.** A *container* is a pair $C = (S, P)$ where $S$ is a type of *shapes* and $P : S \to \mathrm{Type}$ is a family of *positions*. Its *extension* is the polynomial functor

$$C(X) = \sum_{s:S} (P(s) \to X).$$

Container morphisms $(f, f^\sharp) : C \to D$ consist of a shape map $f : S_C \to S_D$ and a contravariant position map $f^\sharp_s : P_D(f\,s) \to P_C(s)$. The representation theorem establishes: container morphisms $C \to D$ are in bijection with natural transformations $C \Rightarrow D$.

**Definition 2.2** (Directed container)**.** A *directed container* extends a container $(S, P)$ with three operations:

- a *root* $\mathrm{o} : \prod_{s:S} P(s)$;
- a *sub-shape* $\downarrow : \prod_{s:S} P(s) \to S$;
- a *shift* $\oplus : \prod_{s:S} \prod_{p:P(s)} P(s\downarrow p) \to P(s)$;

subject to five laws:

**D1** $s\downarrow \mathrm{o}(s) = s$
**D2** $\mathrm{o}(s) \oplus q = q$ (transported along D1)
**D3** $p \oplus \mathrm{o}(s\downarrow p) = p$
**D4** $s\downarrow(p \oplus q) = (s\downarrow p)\downarrow q$
**D5** $(p \oplus q) \oplus r = p \oplus (q \oplus r)$ (transported along D4)

**Theorem 2.3** (Comonad structure [2])**.** *A directed container $(S, P, \mathrm{o}, \downarrow, \oplus)$ induces a comonad on $C$ with:*

$$\varepsilon\,\langle s, v\rangle = v(\mathrm{o}(s))$$
$$\delta\,\langle s, v\rangle = \langle s,\ \lambda p.\,\langle s\downarrow p,\ \lambda q.\, v(p \oplus q)\rangle\rangle$$

*The three comonad laws follow from D1–D5:*

- *Left counit* $\varepsilon \circ \delta = \mathrm{id}$ *from D1, D2.*
- *Right counit* $C(\varepsilon) \circ \delta = \mathrm{id}$ *from D3.*
- *Coassociativity* $C(\delta) \circ \delta = \delta \circ \delta$ *from D4, D5.*

*All five laws and all three comonad laws are formalised sorry-free in Lean 4 (files* `Directed.lean`*,* `CoKleisli.lean` *in the trust-boundaries library).*

# 3 The social graph as a directed container

We now instantiate the abstract framework for social networks.

**Definition 3.1** (Social graph container)**.** Let $\mathcal{G}$ be the directed container defined by:

- $S =$ the type of *social graphs*: a set of users $U$ together with a follow relation $F \subseteq U \times U$ and a distinguished viewer $v_0 \in U$.
- $P(s) = U_s$, the set of users in graph $s$.
- $\mathrm{o}(s) = v_0$, the logged-in viewer.
- $s{\downarrow}p = \mathrm{ego}(s, p)$, the *ego network* of user $p$ in graph $s$: the subgraph induced by $p$ and all users $p$ follows.
- $p \oplus q$: given $p \in P(s)$ and $q \in P(s{\downarrow}p)$, the user $q$ (which is also a user in $s$, since $P(s{\downarrow}p) \subseteq P(s)$).

**Proposition 3.2** (D1–D5 for social graphs)**.** *The social graph container satisfies the five directed container laws:*

***D1*** $\mathrm{ego}(s, v_0) = s$*: the ego network of the viewer is the entire visible graph. (Holds when the viewer can see everyone they follow—the public-posts assumption.)*

***D2*** $v_0 \oplus q = q$*: navigating from the viewer to position* $q$ *in the viewer's ego network yields* $q$*. (Immediate from the definition of* $\oplus$ *as inclusion.)*

***D3*** $p \oplus \mathrm{o}(\mathrm{ego}(s, p)) = p \oplus p = p$*: navigating from* $p$ *to* $p$*'s own root position returns to* $p$*. (The root of* $\mathrm{ego}(s, p)$ *is* $p$ *itself.)*

***D4*** $\mathrm{ego}(s, p \oplus q) = \mathrm{ego}(\mathrm{ego}(s, p), q)$*: the ego network of* $q$ *in the full graph equals the ego network of* $q$ *within* $p$*'s ego network. (Holds when follow relations are global—*$q$*'s follows don't depend on who's looking.)*

***D5*** *Shift associativity:* $(p \oplus q) \oplus r = p \oplus (q \oplus r)$*. (Both sides equal* $r$*, since* $\oplus$ *is inclusion.)*

*Remark* 3.3. D4 is the most substantive law for social networks. It says that the ego network is *context-independent*: what user $q$ can see doesn't depend on which larger graph you computed $q$'s ego network within. This holds for public, asymmetric-follow networks (like early Twitter). It *fails* when privacy settings make visibility context-dependent—which is exactly where the presheaf layer (Section 4) takes over.

**Corollary 3.4** (Social feed as comonadic extract)**.** *By Theorem 2.3, the social graph comonad has:*

$$\varepsilon\,\langle s, v\rangle = v(v_0) \qquad \text{(“my content”—the viewer's own posts)}$$

$$\delta\,\langle s, v\rangle = \langle s,\ \lambda p.\,\langle \mathrm{ego}(s, p),\ \lambda q.\,v(q)\rangle\rangle \qquad \text{(“feed of feeds”)}$$

*The* feed algorithm *is a coKleisli morphism* $f : \mathcal{G}(X) \to Y$*—a function that inspects a local neighbourhood and produces a summary. Applying it everywhere via* extend $f$ *produces each user's personalised feed.*

## 3.1 Haskell encoding

The Haskell types mirror the Lean definitions:

```haskell
-- Domain types
newtype UserId = UserId Int
data User = User { userId ::UserId, handle ::Text, displayName ::Text }
data Post = Post { postId ::PostId, author ::UserId, body ::Text, createdAt ::UTCTime }

-- The social graph
data SocialGraph = SocialGraph
  { users   :: Set UserId
  , follows :: Set (UserId, UserId) -- (follower, followed)
  , viewer  :: UserId
  }

-- Extension functor
data Ext x = Ext SocialGraph (UserId →x)
  deriving Functor

-- Directed container operations
root :: SocialGraph →UserId
root = viewer

egoNetwork :: SocialGraph →UserId →SocialGraph
egoNetwork g u = SocialGraph
  { users   = Set.insert u (followedBy u g)
  , follows = Set.filter (\(a,_) →a == u) (follows g)
  , viewer  = u
  }

shift :: SocialGraph →UserId →UserId →UserId
shift _ _ q = q -- inclusion: q is already a user in the parent graph

-- Comonad operations (derived, not hand-written)
extract :: Ext x →x
extract (Ext g v) = v (root g)

duplicate :: Ext x →Ext (Ext x)
duplicate (Ext g v) = Ext g (\p →Ext (egoNetwork g p) (\q →v (shift g p q)))

extend :: (Ext x →y) →Ext x →Ext y
extend f (Ext g v) = Ext g (\p →f (Ext (egoNetwork g p) (\q →v (shift g p q))))
```

### 3.2 Lean proof obligations

The abstract comonad laws are already proved in the trust-boundaries library. The only new proof work is instantiating D1–D5 for the social graph:

```
-- New file: SocialGraph.lean
import Containers.Directed

def SocialGraph : DirectedContainer where
  Shape := GraphShape        -- users + follow relation + viewer
  Pos   := fun g => g.Users
  root  := fun g => g.viewer
  sub   := fun g u => egoNetwork g u
  shift := fun g _ q => q    -- inclusion
  d1    := by intro g; ext; simp [egoNetwork] -- ego of viewer = full graph
  d2    := by intro g q; rfl                  -- root shift = id
  d3    := by intro g p; rfl                  -- shift to sub-root = id
  d4    := by intro g p q; ext; simp [egoNetwork] -- ego nesting
```

```
  d5    := by intro g p q r; rfl            -- shift associativity (trivial)
```

# 4 Presheaves: the permission layer

The directed container gives us the feed. But a real social network must answer: *which content is visible to which viewer?* This is the domain of presheaves.

**Definition 4.1** (Category of contexts)**.** Let **Ctx** be the category where:

- Objects are *viewing contexts*: triples $(u, t, \pi)$ where $u$ is a user, $t$ is a timestamp, and $\pi$ is a permission set (e.g., {public, follower, admin}).
- A morphism $c_1 \to c_2$ exists when $c_2$ is a *restriction* of $c_1$: same or later time, same or fewer permissions, same or more restricted viewer.

Composition is transitive restriction. Identity is the trivial restriction.

**Definition 4.2** (Content presheaf)**.** A *content presheaf* is a functor $F : \mathbf{Ctx}^{\mathrm{op}} \to \mathbf{Set}$. For each context $c$, $F(c)$ is the set of content visible in that context. Contravariance means: if $c_1 \to c_2$ is a restriction (forgetting privilege), then $F(c_2 \to c_1) : F(c_1) \to F(c_2)$ maps the larger visible set into the smaller one.

*Remark* 4.3 (Privacy bugs as functoriality failures). A privacy bug is a violation of functoriality: a case where restricting privilege does not correctly restrict visible content. For example, if a user blocks another but the blocked user can still see their posts through a cached API response, the restriction morphism has failed to propagate. The presheaf framework makes this precise: every privacy invariant is a functoriality condition.

## 4.1 Haskell encoding

```haskell
-- Viewing context
data Context = Context
  { ctxViewer :: UserId
  , ctxTime   :: UTCTime
  , ctxPerms  :: PermissionSet
  }

-- Permission levels (ordered by privilege)
data Permission = Public | Follower | Mutual | Admin
  deriving (Eq, Ord)

-- Context morphism: evidence that c2 is a restriction of c1
data Restriction = Restriction
  { from :: Context
  , to   :: Context
  }

-- Content presheaf: contravariant functor from contexts to sets of content
class ContentPresheaf f where
  view     :: Context →f content →Set content
  restrict :: Restriction →Set content →Set content
  -- Laws (enforced in Lean):
  --   restrict id = id
  --   restrict (g . f) = restrict f . restrict g
```

### 4.2 Lean formalisation

```
structure Context where
  viewer : UserId
  time   : Nat       -- discrete timestamp
  perms  : Finset Permission

-- Restriction: c1 has at least as much privilege as c2
def Restricts (c1 c2 : Context) : Prop :=
  c2.perms <= c1.perms /\ c1.time <= c2.time

structure ContentPresheaf where
  Content : Type
  view : Context -> Finset Content
  map_restrict : forall c1 c2, Restricts c1 c2 -> view c1 >= view c2
  -- Contravariance: more privilege shows more content
```

## 5 The sheaf condition and controlled failure

A presheaf $F : \mathbf{Ctx}^{\mathrm{op}} \to \mathbf{Set}$ satisfies the *sheaf condition* with respect to a coverage $\{c_i \to c\}$ when compatible local sections glue to a unique global section. In social network terms: if every user's local view of a piece of content is "the same," there is a single global truth.

**Definition 5.1** (Compatibility)**.** Given a cover $\{c_i\}_{i \in I}$ of a context $c$, a family of local sections $\{s_i \in F(c_i)\}$ is *compatible* when for every pair $i, j$ with overlap $c_{ij} = c_i \times_c c_j$:

$$F(c_i \leftarrow c_{ij})(s_i) = F(c_j \leftarrow c_{ij})(s_j).$$

**Definition 5.2** (Consistency classification)**.** We classify content types by their sheaf behaviour:

| Level | Meaning | Examples |
|---|---|---|
| Sheaf | All viewers agree | Public post text, user profiles |
| EventualSheaf | Viewers converge | Like counts, follower counts |
| Presheaf | Deliberately inconsistent | Personalised feeds, blocked content |

**Theorem 5.3** (Public posts form a sheaf)**.** *Let $F$ be the content presheaf restricted to public posts (posts with no access restrictions). Then $F$ satisfies the sheaf condition: for any cover $\{c_i\}$ and compatible local sections, there exists a unique global section.*

*Proof sketch.* A public post has the same text, author, and timestamp regardless of viewer. The restriction maps are all identities on the content (only the *set* of visible posts changes, not the posts themselves). Compatibility is automatic. □

**Theorem 5.4** (Blocked content violates the sheaf condition)**.** *There exist contexts $c_1, c_2$ and content $p$ such that $p \in F(c_1)$ and $p \notin F(c_2)$, with no global section compatible with both.*

*Proof sketch.* Let Alice post $p$. Let $c_1$ be Alice's own context (she can see her posts) and $c_2$ be the context of a user Alice has blocked. Then $p \in F(c_1)$ and $p \notin F(c_2)$. Any global section either includes or excludes $p$, contradicting one of the local sections. □

*Remark* 5.5 (The architectural principle). A well-designed social network makes a *deliberate choice* for each content type about where in the classification of Definition 5.2 it sits. The engineering complexity comes from managing the boundary between sheaf and presheaf content within the same system. The categorical framework makes this boundary explicit and amenable to formal verification.

### 5.1 Haskell encoding

```haskell
-- Consistency levels as a type-level tag
data ConsistencyLevel = Sheaf | EventualSheaf | Presheaf

-- Tag each content type with its consistency guarantee
type family Consistency (a ::Type) ::ConsistencyLevel where
  Consistency PublicPost  = 'Sheaf
  Consistency UserProfile = 'Sheaf
  Consistency LikeCount   = 'EventualSheaf
  Consistency FollowerCount = 'EventualSheaf
  Consistency PersonalFeed = 'Presheaf
  Consistency BlockedContent = 'Presheaf

-- The type system prevents treating Presheaf content as Sheaf content:
-- you cannot cache or share PersonalFeed across viewers without a
-- type error.
```

## 6 Interaction: comonad meets presheaf

The directed container comonad (Section 3) and the content presheaf (Section 4) interact through the feed computation. The feed is the comonadic `extend` applied to a presheaf-filtered content stream.

**Definition 6.1** (Filtered feed)**.** Given a feed algorithm $f : \mathcal{G}(\text{Post}) \to [\text{Post}]$ and a content presheaf $F$, the *filtered feed* for context $c$ is:

$$\text{feed}(c) = f\big(\langle \text{ego}(s, \text{viewer}(c)),\ \lambda p.\, F(c)(p) \rangle\big)$$

where $F(c)(p)$ returns the posts by user $p$ visible in context $c$.

*Remark* 6.2 (Colimits don't commute with filtering). A naïve implementation might compute the feed (a colimit of content sources) and then filter by permissions. But extend does not commute with presheaf restriction in general: filtering after ranking may produce different results than filtering before ranking, since the ranking function sees a different neighbourhood. This non-commutativity is a source of real bugs in social network implementations (e.g., a "top post" being filtered out after ranking, leaving a gap).

## 7 The minimal social network

We identify the *irreducible core* of a social network: the minimum feature set that exhibits network effects.

**Definition 7.1** (Minimal social network)**.** A minimal social network consists of:

1. **Identity**: a user type with handle and display name.
2. **Asymmetric connection**: a follow relation $F \subseteq U \times U$ (directed, no mutual consent required).
3. **Broadcast**: a post type (text, author, timestamp).
4. **Chronological feed**: the comonadic extract applied to a reverse-chronological ordering.

The data model is three tables:

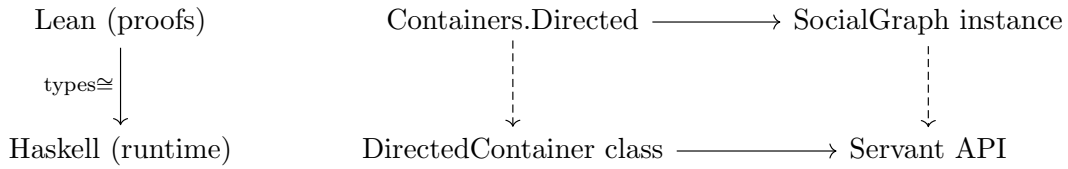
| Table | Columns |
|---|---|
| `users` | `id`, `handle`, `display_name`, `bio`, `created_at` |
| `follows` | `follower_id`, `followed_id`, `created_at` |
| `posts` | `id`, `author_id`, `body` (text, $\leq 280$ chars), `created_at` |

The feed query is a single join:

```
SELECT posts.* FROM posts
  JOIN follows ON posts.author_id = follows.followed_id
  WHERE follows.follower_id = :me
  ORDER BY posts.created_at DESC
  LIMIT 50;
```

*Remark* 7.2 (Categorical simplification). The minimal social network satisfies the full sheaf condition: all content is public, all viewers see the same posts (differing only in which users they follow). The permission presheaf is trivial (the constant presheaf), and the comonad is the entire story. This is the base case from which complexity is added incrementally.

# 8 Implementation architecture




The implementation proceeds in layers:

1. **Haskell domain types**: algebraic data types for `User`, `Post`, `SocialGraph`, `Context`.
2. **Haskell directed container**: `root`, `egoNetwork`, `shift` implementing the comonad.
3. **Lean instantiation**: prove D1–D5 for these definitions, importing the trust-boundaries library. Comonad laws follow for free.
4. **Servant API**: six type-safe HTTP routes wiring the comonad into a web service.
5. **Presheaf/sheaf layer**: added when the network supports non-public content.

The Haskell–Lean correspondence is nearly mechanical:

| **Haskell** | **Lean** |
|---|---|
| `data T = A \| B \| C` | `inductive T \| A \| B \| C` |
| `class C a where` | `class C (a : Type) where` |
| `instance C T where` | `instance : C T where` |
| `newtype F a = F (a -> b)` | `def F (a : Type) := a -> b` |
| type constraint | `theorem` / `Prop` |

# 9 Future directions

1. **Zappa–Szép products**: the trust-boundaries library includes Zappa–Szép products of directed containers (`ZappaSzep.lean`). In the social network setting, this could model the interaction between two independent graph structures (e.g., follow graph and group membership graph) whose directed containers compose non-trivially.

2. **CoKleisli morphisms as feed algorithms**: different feed algorithms (chronological, ranked, "following" vs "for you") are different coKleisli morphisms. The coKleisli category gives a principled space of feed algorithms, with composition corresponding to pipeline stages.

3. **Controlled sheaf failure as a design language**: rather than ad-hoc privacy rules, express each new feature as a choice of consistency level (Definition 5.2). The Lean proof obligation forces the implementer to verify that the chosen level is achievable.

4. **Cofunctors for cross-network interaction**: container morphisms between different social network instances (federation, cross-posting) are cofunctors between directed containers, with the trust-boundaries transport theorem guaranteeing that trust levels translate safely.

## Acknowledgements

The directed container library and its Lean formalisation were developed in the trust-boundaries project. The categorical perspective on social networks emerged from conversations between Robin Langer and Neil Ghani at Kodamai.

## References

[1] M. Abbott, T. Altenkirch, and N. Ghani. Containers: constructing strictly positive types. *Theoretical Computer Science*, 342(1):3–27, 2005.

[2] D. Ahman, J. Chapman, and T. Uustalu. When is a container a comonad? *Foundations of Software Science and Computation Structures (FoSSaCS)*, 2012.

[3] R. Langer. Trust boundaries: epistemically honest AI agents via directed container comonads. Technical report, 2026.

[4] T. J. Green, G. Karvounarakis, and V. Tannen. Provenance semirings. *PODS*, 2007.